SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Assignment Title: Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

Nature of Assignment: Individual

Name: Dharshavardhan.SB

Reg: 192420039

Duration: One week

Total Marks: 50

Submission Date: _____

Scenario – Agile Delivery Planning for the Order Fulfilment Platform

A retailer sells through a website, a mobile app and twelve physical stores. It plans an integrated order fulfilment platform to reduce failed orders, slow refunds and unclear order ownership. Warehouse pickers prioritize accurate stock reservation and pick lists, support agents prioritize order visibility and fast return handling, and finance reviewers require auditable refunds and payment reconciliation. A seven-member team must select a feasible eight-week release before a fixed festival-sale date despite uncertain capacity, limited stakeholder time and delayed payment-gateway access. Create and validate an Agile delivery strategy connecting these priorities to a justified backlog, team responsibilities, Lean flow controls and sprint goals. Revise the strategy when regions, staffing and coordination needs change, while protecting a complete order-to-delivery-to-refund journey.

Planning conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Checkout, stock reservation, payment, picking, dispatch, tracking, returns, refunds and support lookup
Users	Customer, warehouse picker, support agent, store manager, finance reviewer and administrator
Reference data	Synthetic products, synthetic orders and simulated stock movements; no real customer or card data
Integration constraint	Payment gateway sandbox may be unavailable; preserve local order records and retry payment requests safely without double charging
Validation need	Verify stock, order status, price, authorization, persistence and refund audit behaviour

The retailer proposes an omnichannel order fulfilment platform. Its intended modules are a storefront catalogue, cart and checkout, payment integration, inventory synchronization, order tracking, a warehouse picking dashboard, returns and refunds, a support console and sales analytics. The system should connect a customer order to stock reservation, payment confirmation, picking, dispatch, tracking, delivery and, where needed, an authorized return and refund.

A seven-member cross-functional team plans an initial release within eight weeks using four two-week sprints. There is no established velocity. Test access to the payment gateway sandbox begins in week four, and a barcode-scanner test device arrives in week five. Warehouse and finance stakeholders can attend clarification and review meetings for ninety minutes per week. These are assignment planning assumptions.

Principal users are customers, warehouse pickers, support agents, store managers, finance reviewers and administrators. They need different permissions. Pickers require accurate pick lists; support agents require readable order and return status; finance reviewers require auditable refund approvals; engineers require traceable links between orders, payments and releases. Automation must preserve human responsibility for refund approval and stock corrections.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, stock counts and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live inventory control or secure payment handling. Verify claims through repeatable checks. Distinguish deterministic rules, such as fixed low-stock thresholds, from trained machine-learning forecasting, and label simulations and proposed extensions accurately.

Design Challenge:

Management requests all nine modules in the initial release. Pickers prioritize stock reservation, pick lists and dispatch; support agents prioritize order tracking and return handling; finance reviewers prioritize refund approval and payment reconciliation. Gateway access and limited review time constrain delivery. A broad release may leave every workflow incomplete, while a storefront-only release may fail to reduce failed or delayed orders.

Design, simulate and validate an Agile project delivery strategy using a project-management tool of your choice that produces a coherent and achievable first release under these conditions.

Develop alternative delivery approaches and create a justified strategy connecting backlog priorities to the identified operational problems. Establish team responsibilities, capacity assumptions, dependency sequencing, stakeholder review arrangements and Lean practices for controlling waiting, rework and work in progress.

Investigate at least two suitable project-management tools, such as Jira, Trello, Azure DevOps or GitHub Projects. Select one and justify its suitability for your strategy, considering planning, workflow management, collaboration, reporting, accessibility and cost. Explain any limitations and how you will address them. Before simulation, predict the principal delivery bottleneck and define measurable criteria for judging the strategy. Use the resulting evidence to revise your plan and defend the features retained, deferred or resequenced. Your final design must explain how the team will deliver, why the selected release should improve fulfilment, and how the chosen tool supports the strategy.

TASK 1 – Investigate the Problem and Define Delivery Outcomes 5 Marks

Investigate the platform from the perspectives of customers, warehouse pickers, support agents, finance reviewers and the development team.

- Formulate at least three questions to investigate about priorities, workflow problems or delivery constraints.
- Obtain initial evidence through a small stakeholder discussion, questionnaire or documented review of the existing process. If access is unavailable, use clearly identified simulated stakeholder responses.
- Define the product goal, first-release scope and exclusions.
- Establish at least three measurable delivery-success criteria.
- Examine technical, operational and financial feasibility at a level appropriate to project planning.

Evidence Expected

Investigation questions, stakeholder findings or labelled simulated responses, a problem statement, scope boundaries, feasibility observations and measurable success criteria.

TASK 2 – Design and Justify the Project Management Approach 6 Marks

Develop two meaningfully different delivery approaches for the scenario. Consider Scrum, Lean flow management, XP practices or a justified combination.

- For each approach, explain how work would be selected, coordinated and reviewed.
- Identify its suitability for changing requirements and cross-functional work.
- Examine likely limitations, overheads and delivery risks.
- Select an approach and create a tailored operating model defining responsibilities, planning activities, review points, escalation procedures and stakeholder participation.
- Explain how Lean principles will help reduce waiting, unnecessary handovers, rework or excessive work in progress.

Evidence Expected

Comparison of two approaches, a justified selection and an original operating model with explicit Lean practices.

TASK 3 – Construct a Prioritized Product Backlog in the Selected Tool 8 Marks

Create a project in your selected management tool and develop a backlog containing:

- At least four epics and twelve user stories covering order management, inventory and picking, payments and refunds, and tracking or support. Justify which modules are retained or deferred.
- User stories with clear acceptance criteria.
- Priorities, estimates, dependencies and responsible roles.
- A justified prioritization method.
- A distinction between essential first-release features and deferred features.
- A Definition of Ready and a Definition of Done appropriate to the project. Incorporate relevant privacy, accessibility and quality expectations into the backlog or completion criteria.
- An explanation of at least two decisions where competing stakeholder requests influenced your priorities.

Evidence Expected

Selected-tool backlog screenshots or export, story acceptance criteria, prioritization rationale, dependencies, and Definitions of Ready and Done.

TASK 4 – Create a Capacity-Based Release Plan and Delivery Workflow 7 Marks

Design the four-sprint release plan using explicit assumptions about team availability and capacity.

- Define a goal for each sprint.
- Select work that fits the assumed capacity.
- Explain your estimation method and avoid treating story points as fixed hours.
- Represent dependencies and cross-functional responsibilities, including the week-four gateway and week-five scanner constraints.
- Create a development workflow in the selected tool with statuses, transition rules and completion checks. Distinguish this workflow from the operational order lifecycle.
- Set and justify work-in-progress limits where applicable.
- Define how blocked work, urgent requests and unfinished stories will be handled.
- Show how the selected features collectively achieve the product goal.

Evidence Expected

Release plan, capacity assumptions, sprint goals, selected-tool board or workflow, work-in-progress policies and dependency-management decisions.

TASK 5 – Simulate Delivery and Validate Your Design 7 Marks

Simulate the first sprint using a documented sequence of at least ten work events, such as starting a story, identifying a dependency, encountering a defect, receiving clarification or completing a review.

- Before simulation, predict where a bottleneck is most likely to occur.
- Identify which planning or workflow rule should help control it and what evidence would indicate that the rule is effective.
- Update the selected management tool to reflect the simulated events.
- Record suitable indicators such as completed versus planned work, blocked time, cycle time, work in progress or reopened stories.
- Use the results to identify a limitation in your original plan and create an improvement for the next sprint.

Evidence Expected

Prediction, event log, management-tool updates, at least three delivery indicators, interpretation of results and a revised planning or workflow decision.

All simulated results must be clearly labelled. Do not present simulated activity as actual team delivery.

TASK 6 – Redesign Delivery Under Changed Conditions 12 Marks

Changed Requirement

At the end of the first simulated sprint, the organization introduces the following changes:

- Two additional regional warehouses must use the platform.
- A second seven-member development team joins the project.
- The payments engineer from the original team becomes unavailable for the remaining period.
- Region-specific tax, shipping and refund-approval rules are now required.
- The festival-sale release deadline remains unchanged.

Create a revised strategy that coordinates both teams, preserves a useful first release and manages the new constraints.

Component	Required Work	Marks
A. Diagnose the impact	Identify affected assumptions, capacity, scope, dependencies and risks using evidence from your original plan.	2
B. Define redesign objectives	Establish measurable objectives for the revised strategy, including achievable scope and coordination expectations.	2
C. Create the revised strategy	Redesign backlog priorities, team responsibilities, release sequencing and dependency coordination. Incorporate justified scaled Agile and Lean practices.	4
D. Validate the redesign	Simulate at least five additional events and compare the revised strategy with the original using equivalent indicators.	2
E. Defend the decision	Explain benefits, limitations and trade-offs. Show why the revised strategy remains feasible under the unchanged deadline.	2

You may use selected SAFe-inspired coordination practices or another justified scaled Agile approach. Explain what you adopt and why it fits this project.

Evidence Expected

Impact analysis, measurable redesign objectives, revised management-tool artefacts, cross-team coordination design, comparative simulation evidence and an evidence-based defence.

TASK 7 – Individual Technical Reflection 5 Marks

Explain:

- Which project-management decision you consider most significant and why.
- How your evidence influenced a change to the original strategy.
- Where your final design remains vulnerable.
- How you addressed an ethical concern involving privacy, workload, accessibility or transparency.
- What you would investigate next before using the strategy with a real development team.

Evidence Expected

A concise individual reflection demonstrating understanding of the created strategy and its limitations. Refer to your own artefacts and findings.

MARK DISTRIBUTION

Task	Assessed Area	Marks
Task 1	Problem investigation and measurable delivery outcomes	5
Task 2	Original Agile and Lean operating model	6
Task 3	Prioritized Jira backlog and acceptance criteria	8
Task 4	Capacity-based release plan and workflow	7
Task 5	Delivery simulation and evidence-based improvement	7
Task 6	Day-4 Design Challenge – scaled delivery redesign and validation	12
Task 7	Individual technical reflection	5
TOTAL		50

Submission Requirements

- Assignment report covering investigation findings, scope, delivery approaches and project-management tool selection.
- Project export or screenshots showing epics, user stories, acceptance criteria, priorities, estimates and dependencies.
- Definition of Ready, Definition of Done, release plan, capacity assumptions and selected-tool workflow with work-in-progress limits.
- Sprint 1 simulation: prediction, event log, management-tool updates and delivery indicators, all clearly labelled as simulated.
- Day-4 redesign artefacts: impact analysis, redesign objectives, revised backlog, cross-team coordination design and comparative simulation evidence.
- Individual reflection and acknowledgement of sources and tools used.

Assessment Verification

Students may be asked to explain a prioritization decision, trace a story through the selected workflow, justify capacity or a work-in-progress limit, interpret an indicator, or revise the plan when a constraint changes.

Overall Assessment Expectation

Investigate → Design → Plan → Simulate → Validate → Redesign → Defend

The final submission must demonstrate an original, evidence-supported delivery strategy aligned with CO1, BTL 5 – Evaluate and BTL 6 – Create.

ASSIGNMENT 1 – ANSWERS

CSA10 – SOFTWARE ENGINEERING

Assignment Title: Design, Simulate and Validate an Agile Delivery Strategy for the Omnichannel Order Fulfilment Platform

Planning note: The stakeholder findings, estimates, sprint events and metrics below are explicitly simulated planning assumptions prepared for this assignment. They are not claims of actual interviews, Jira activity or production performance.

TASK 1 – INVESTIGATE THE PROBLEM AND DEFINE DELIVERY OUTCOMES (5 MARKS)

1.1 Investigation questions

Which order-stage information do customers and support agents need to see to resolve delayed or failed orders?

What stock reservation and picking information do warehouse teams need to prevent accepting orders that cannot be fulfilled?

What controls and evidence must finance reviewers see before a refund is approved and reconciled?

Which external dependencies, especially payment-gateway access and barcode-scanner availability, could delay the eight-week release?

1.2 Simulated stakeholder findings

Stakeholder	Simulated finding	Delivery implication
Customer	Needs reliable confirmation, tracking and clear refund status.	Provide one consistent order timeline and customer-visible status.
Warehouse picker	Needs reserved stock, accurate pick lists and dispatch confirmation.	Prioritize inventory reservation and a basic picking dashboard.
Support agent	Needs a searchable order record and return status.	Provide support lookup with order, payment and return history.
Finance reviewer	Needs refund authorization, reason and audit trail.	Keep human approval and record every refund action.
Development team	Gateway access starts in week 4; scanner arrives in week 5.	Build a payment adapter and manual picking fallback before integrations.

1.3 Problem statement and product goal

Problem statement: Orders move across the website, app and twelve stores without a sufficiently connected view of stock, payment, fulfilment and returns. This can cause failed orders, delayed refunds and unclear ownership. The first release must connect the essential order journey while respecting limited capacity and delayed integration access.

Product goal: Deliver, within eight weeks, a testable minimum viable platform that records an order, reserves stock, records payment state, supports picking and dispatch, displays order status, and manages an authorized return/refund with an auditable history.

1.4 First-release scope and exclusions

Included in first release	**Deferred or excluded**
Catalogue and basic cart/checkout using synthetic products and orders.	Advanced recommendations and personalized storefront.
Order record, stock reservation and low-stock visibility.	Full real-time synchronization with all twelve stores.
Payment sandbox adapter, safe retry and duplicate-charge protection; local simulated payment state when sandbox is unavailable.	Production payment processing until gateway certification and security review.
Basic picking list, dispatch update and order tracking.	Automated barcode workflows until device testing in week 5.
Support lookup, return request, finance-authorized refund and audit log.	Automated refund approval and advanced analytics.

1.5 Measurable success criteria

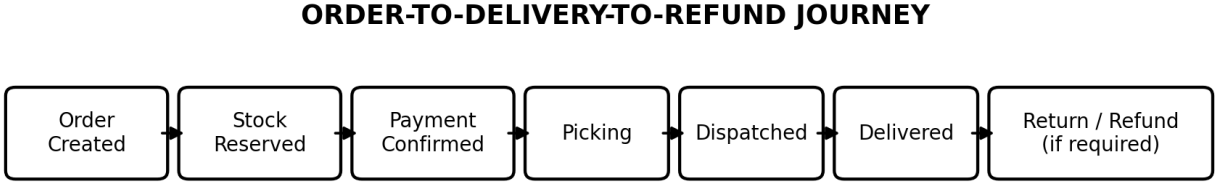


Figure 1. Operational order fulfilment lifecycle (proposed).

Measure	Target for simulated acceptance	Verification
Order journey completeness	At least 95% of test orders move through order creation, stock decision, payment state, picking, dispatch and tracking without a missing state.	Run 20 synthetic end-to-end test orders.
Stock reservation	All test reservations prevent a second order from reserving the same available unit.	Run concurrent reservation tests for the same SKU.
Payment retry safety	Repeated retry requests create no duplicate successful charge in the simulated test set.	Replay the same payment request and inspect transaction IDs.
Refund auditability	100% of test refunds record approver, reason, amount, timestamp and linked order/payment.	Inspect refund audit records.
Support lookup	A support agent can locate a test order and see its current fulfilment and return status within 30 seconds.	Timed usability check using 10 synthetic orders.

1.6 Feasibility

Technical: Feasible as a limited release using modular services, synthetic data, a payment adapter and a local retry queue. Integration and security risks remain until the gateway and scanner are tested. Operational: Feasible if warehouse and finance representatives provide one 90-minute weekly clarification/review session and a manual picking fallback is retained. Financial: A small team and an existing project-management tool keep initial costs controlled; exact monetary cost requires staff-rate and hosting estimates that are not supplied in the scenario.

TASK 2 – DESIGN AND JUSTIFY THE PROJECT MANAGEMENT APPROACH (6 MARKS)

Approach	How it works	Advantages	Limitations
Approach A: Scrum with Lean flow and XP	Four two-week sprints; sprint planning, daily coordination, review and retrospective. Lean WIP	Clear sprint goals, regular stakeholder feedback, suitable for changing	Fixed sprint boundaries can delay urgent work; ceremonies consume

engineering practices	limits and XP pairing, automated tests and continuous integration are used.	requirements and cross-functional delivery.	time; velocity is unknown initially.
Approach B: Kanban/Lean flow with continuous prioritization	Work is pulled continuously through a visual board; priorities can change whenever capacity becomes available; replenishment and service reviews are held weekly.	Flexible for uncertain dependencies, exposes blocked work and limits multitasking.	Less predictable release scope; difficult to coordinate a fixed eight-week release without explicit milestones.

Selected approach: Scrum supported by Lean flow controls and selected XP practices. The fixed festival deadline and four two-week iterations need timeboxed goals, while Lean controls reduce waiting and XP practices protect quality.

2.1 Tailored operating model

Role	**Main responsibility**
Product Owner (one team member)	Owns product goal, backlog ordering, acceptance decisions and stakeholder communication.
Scrum Master / delivery facilitator (one team member)	Facilitates ceremonies, removes blockers, monitors WIP and escalates unresolved dependencies.
Developers (three team members)	Implement order, inventory, tracking and support functions; review code and maintain integration.
QA / test engineer (one team member)	Defines test cases, automates regression tests and verifies acceptance criteria.
Payments/integration engineer (one team member)	Builds payment adapter, retry/idempotency controls and payment/refund integration.

The seven roles may share technical tasks, but each accountability has one named owner in the project board. The Product Owner gathers questions before the weekly 90-minute stakeholder slot. Sprint planning and review are timeboxed; daily team coordination is 15 minutes. Unresolved blockers lasting

more than one working day are escalated to the facilitator and Product Owner. Security, finance approval and stock correction decisions remain with authorized humans.

2.2 Lean practices

Limit active development to 3 stories and active testing to 2 stories; finish work before starting more.
Use a visible blocked flag with blocker owner, reason and next action; review blocked work daily.
Automate build, unit tests and regression checks to detect defects early and reduce rework.
Prepare acceptance criteria before sprint planning to reduce clarification waiting.
Use small stories and vertical slices to reduce handovers and deliver a complete order journey incrementally.
Track cycle time, blocked time, reopened stories and WIP; improve the largest source of delay at each retrospective.

TASK 3 – CONSTRUCT A PRIORITIZED PRODUCT BACKLOG (8 MARKS)

Selected tool: Jira. A Scrum project will contain epics, stories, estimates, assignees, dependencies, acceptance criteria and sprint boards. The backlog below is a proposed Jira-ready export; it is not a screenshot or evidence that a live Jira project has already been created.

3.1 Tool comparison and selection

Tool	Suitability	Limitations and response
Jira	Strong backlog, epic/story hierarchy, sprint planning, workflows, dependencies, dashboards and team reporting; commonly used for Agile projects.	Configuration can be complex and some features require paid plans. Use a simple workflow, limited fields and the available free/education tier if eligible.
Trello	Simple visual boards, cards, labels and collaboration; quick to learn and suitable for a small flow.	Less structured sprint, dependency and estimation reporting without add-ons. Use checklists and labels, but maintain a separate release metrics sheet if selected.

Jira is selected because this assignment requires four sprints, a prioritized backlog, story estimates, dependencies, WIP monitoring and sprint reporting. Access and cost must be confirmed before real use; the design can be transferred to Trello if Jira access is unavailable.

3.2 Prioritized backlog

Prioritization method: MoSCoW, supported by dependency order and risk reduction. Must-have stories establish the minimum complete order-to-delivery-to-refund journey. Should-have items improve usability and resilience; could-have analytics and advanced automation are deferred.

ID / Epic	User story and acceptance criteria	Priority / points	Dependency / owner
E1 Order management – US01	As a customer, I can create an order from a cart. Given valid synthetic items and address, when I confirm checkout, then a unique order ID and item/price snapshot are saved.	Must / 3	None; Order developer
E1 – US02	As the system, I can validate price and order totals. Given a cart, when checkout is submitted, then item totals, tax placeholder and grand total are calculated and stored consistently.	Must / 3	US01; Order developer
E1 – US03	As an administrator, I can view and update permitted order states. Invalid state transitions are rejected and every valid change is timestamped.	Must / 5	US01; Order developer
E2 Inventory & picking – US04	As the system, I can reserve available stock. A reservation reduces available-to-promise quantity and prevents overselling under concurrent requests.	Must / 5	US01; Inventory developer
E2 – US05	As a picker, I can view a pick list by order and SKU. The list shows quantity, location placeholder and reservation state.	Must / 3	US04; Inventory developer
E2 – US06	As a picker, I can mark items picked and an order dispatched. Only	Must / 3	US05; Inventory developer

	reserved orders can be picked; dispatch is timestamped.		
E3 Payments & refunds – US07	As the system, I can record payment status against an order. A payment attempt has a unique reference and status even if the sandbox is unavailable.	Must / 5	US01; Payments engineer
E3 – US08	As the system, I can retry a payment safely. Repeating an identical request uses an idempotency key and does not create a second charge.	Must / 5	US07; Payments engineer
E3 – US09	As a finance reviewer, I can approve or reject a refund. Only an authorized reviewer can decide, and reason, amount, approver and timestamp are stored.	Must / 5	US07, US03; Payments engineer + QA
E3 – US10	As finance, I can reconcile payment and refund records. Each transaction links to an order and has a traceable status.	Should / 3	US07, US09; Payments engineer
E4 Tracking & support – US11	As a customer, I can view order status from confirmation to dispatch and delivery. The timeline reflects recorded state changes.	Must / 3	US03, US06; Support developer
E4 – US12	As a support agent, I can search by order ID and view payment, fulfilment and return status, subject to role permissions.	Must / 3	US03, US07, US11; Support developer
E4 – US13	As a customer, I can request a return with an item and reason. The request is linked to the original order and cannot bypass approval.	Should / 3	US03, US12; Support developer

E4 – US14	As an administrator, I can view basic sales and fulfilment counts using synthetic data.	Could / 2	US01, US03, US06; Analyst/developer

3.3 Epic and release decisions

The four epics are Order Management, Inventory and Picking, Payments and Refunds, and Tracking and Support. Catalogue/cart functions are limited to the minimum needed for checkout. Sales analytics, advanced recommendations, full multi-store live synchronization and automated refund approval are deferred so that operational users receive a complete basic workflow rather than nine shallow modules. Trade-off 1: Warehouse requests for accurate reservation and picking take priority over advanced storefront features because incorrect stock creates failed orders. Trade-off 2: Finance's auditable human refund approval is retained before advanced analytics because financial control and traceability are essential.

3.4 Definitions

Definition of Ready (DoR): A story has a clear user/value statement, acceptance criteria, priority, estimate, owner, identified dependencies, test data, and privacy/accessibility considerations. External access assumptions are recorded.

Definition of Done (DoD): Code is reviewed and merged; automated tests pass; acceptance criteria are demonstrated; role permissions and audit behaviour are checked; no critical/high defect remains; documentation and release notes are updated; and the Product Owner accepts the story. Synthetic data only is used.

Quality and ethics: Use least-privilege role access, avoid real customer/card data, mask sensitive values in logs, provide keyboard-accessible controls and readable status text, and keep human approval for refunds and stock corrections.

TASK 4 – CREATE A CAPACITY-BASED RELEASE PLAN AND DELIVERY WORKFLOW (7 MARKS)

4.1 Capacity assumptions and estimation

Assume seven members are available for 6 productive project hours per day over 10 working days per sprint: $7 \times 6 \times 10 = 420$ person-hours gross per sprint. Reserve 20% for meetings, support, integration uncertainty and defects, leaving approximately 336 person-hours of planned capacity. This is a planning assumption, not a measured team velocity. Story points express relative complexity, risk and effort; they are not hours.

Because no velocity exists, Sprint 1 is deliberately conservative. After Sprint 1, the team uses completed points and observed cycle time to recalibrate. Do not convert points into a guaranteed number of hours.

4.2 Four-sprint release plan

Sprint / weeks	Sprint goal	Selected work and dependencies
Sprint 1 / Weeks 1–2	Establish a reliable order and stock foundation.	US01, US02, US04, US03 (initial state rules); create synthetic data and CI tests. Confirm requirements with warehouse and finance.
Sprint 2 / Weeks 3–4	Connect payment state and basic fulfilment.	US05, US06, US07 and initial US08. Gateway sandbox access begins in week 4; develop against a mock adapter before access.
Sprint 3 / Weeks 5–6	Complete safe payment retry, tracking and controlled returns.	Finish US08, US09, US11, US12 and US13. Scanner arrives in week 5; test device integration but retain manual picking fallback.
Sprint 4 / Weeks 7–8	Stabilize and validate the complete release journey.	US10, regression/security/accessibility checks, end-to-end tests, defect fixes, operational guide and release readiness review. US14 only if capacity remains.

4.3 Development workflow and policies

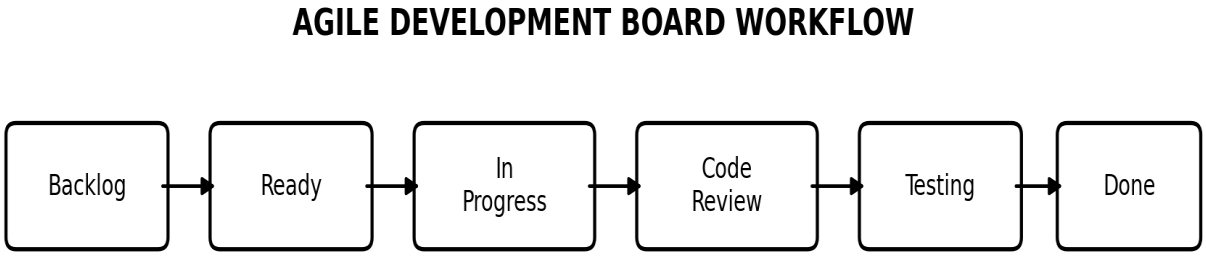


Figure 2. Development workflow used to manage stories.

Workflow status	Entry / exit rule
Backlog	Not yet selected; Product Owner orders work.
Ready	DoR satisfied; dependency and acceptance criteria confirmed.
In Progress	Developer starts work; WIP limit applies.
Code Review	Peer review and automated build/tests required.
Testing	QA verifies acceptance criteria and regression tests.
Blocked	Reason, owner and next action recorded; counted separately from active WIP.
Done	DoD satisfied and Product Owner acceptance recorded.

This is the development workflow, not the operational order lifecycle. The order lifecycle is: Created → Stock Reserved → Payment Pending/Confirmed/Failed → Picking → Dispatched → Delivered → Return Requested → Refund Pending Approval → Refunded/Rejected.

WIP limits: maximum 3 stories In Progress and 2 in Testing. A blocked story remains visible and is not hidden in the backlog. The team swarms to clear blockers before pulling new work. Urgent requests are assessed by the Product Owner; only a genuine release-critical item can displace planned work, with scope removed to preserve capacity. Unfinished stories return to the backlog, are re-estimated if needed and are not automatically counted as completed.

Dependency sequencing: order creation precedes stock reservation; reservation precedes picking; payment state is linked to the order; refund requires payment/order traceability and human approval; tracking consumes order and dispatch events. A mock payment adapter is used until week 4, and manual pick confirmation remains available until scanner validation in week 5.

TASK 5 – SIMULATE DELIVERY AND VALIDATE YOUR DESIGN (7 MARKS)

Predicted bottleneck: payment-gateway access and clarification of payment/refund rules are most likely to block integration. The control is to build against a mock adapter, define idempotency behaviour early, record blockers with owners, and limit WIP. Evidence of effectiveness would be reduced blocked time, fewer reopened stories and completed end-to-end tests.

5.1 Sprint 1 simulated event log

Event	Simulated event	Board / process response
1	Sprint planning selects US01, US02, US04 and US03.	Stories moved from Ready to In Progress according to WIP limit.
2	Team confirms synthetic product and order fields.	Acceptance criteria and sample data added to US01.
3	US04 reveals uncertainty about concurrent stock reservation.	US04 marked Blocked; QA prepares concurrency test.
4	Warehouse clarification confirms reservation must prevent overselling.	Acceptance criteria updated; blocker cleared.
5	A defect allows a negative available-stock value.	US04 remains In Progress; defect logged and assigned.
6	Developer adds atomic reservation check.	Unit test added for concurrent reservation.
7	US01 passes order-creation tests.	US01 moved to Done after review and QA.
8	US02 total calculation fails a boundary-value test.	Story returned to In Progress; test case added.
9	US03 state-transition rules are reviewed with support.	Invalid transition cases added to acceptance tests.
10	Sprint review demonstrates order creation and stock reservation; US02 remains unfinished.	Only accepted stories counted as Done; unfinished work returned to backlog.

5.2 Simulated delivery indicators

Indicator	Simulated result	Interpretation
Planned vs completed	4 stories planned; 2 completed (US01 and US04); 2 not completed/accepted (US02 and US03).	50% story completion; planning was optimistic for the first sprint.

Blocked time	US04 blocked for 1 working day pending warehouse clarification.	Stakeholder clarification delay is measurable and can be reduced by preparing questions earlier.
Cycle time	US01 took 3 working days from In Progress to Done.	Use as an initial observation only; one story is not a stable baseline.
Reopened / rework	US02 required one return to development after a boundary test failure.	Earlier test-case design can reduce late rework.
WIP	Peak active development WIP was 3 stories.	The WIP cap prevented additional work from starting while existing work was unfinished.

These figures are simulated and illustrate how evidence should be recorded; they do not represent actual team performance.

5.3 Improvement for Sprint 2

The original plan underestimated clarification and test-design effort. For Sprint 2, hold a short pre-planning refinement with warehouse, finance and support; require test cases for boundary conditions before a story becomes Ready; reserve approximately 15% of sprint capacity for integration/defects; and keep the same WIP limits. Start payment work with a mock adapter before the gateway opens in week 4.

TASK 6 – REDESIGN DELIVERY UNDER CHANGED CONDITIONS (12 MARKS)

6.1 A. Diagnose the impact

Change	**Impact on original plan**	**Risk / response**
Two additional regional warehouses	Stock, shipping, tax and refund rules become region-specific; original single-region assumptions are invalid.	Add region identifier and configurable rule tables; validate with representative regional test cases.

Second seven-member team joins	Nominal capacity doubles, but onboarding and coordination reduce immediate throughput.	Split work by bounded components; shared standards and integration ownership required.
Original payments engineer unavailable	Payment/refund work loses its primary specialist; gateway integration and reconciliation are at risk.	Assign a replacement integration owner from the new team; conduct handover using interface notes and tests.
Region-specific tax, shipping and refund approval	Order totals, delivery options and refund authorization become more complex.	Add rules engine/configuration and region-aware approval matrix; defer advanced analytics.
Deadline unchanged	Scope cannot expand without trade-offs.	Protect core order-to-delivery-to-refund path; defer nonessential features.

The original assumptions of one region, one seven-member team and continuous payment-engineer availability no longer hold. More people do not automatically mean proportionally more delivery because onboarding, shared environments and integration dependencies consume capacity.

6.2 B. Measurable redesign objectives

Support three warehouse regions in the release using configurable region, tax and shipping settings.

Complete one end-to-end test journey per region, including order, reservation, payment state, dispatch, return and refund authorization.

Keep all refund decisions human-approved and auditable for each region.

Maintain the original eight-week deadline by deferring sales analytics, advanced recommendations and nonessential automation.

Keep cross-team integration blockers visible, assign an owner within one working day and review dependencies at least twice weekly.

6.3 C. Revised backlog, responsibilities and scaled coordination

Reprioritization: Add region configuration, regional tax/shipping calculation and region-specific refund approval as Must-have work. Keep core order, reservation, payment idempotency, picking, tracking and support lookup. Defer US14 analytics and advanced barcode automation if testing capacity becomes constrained.

Workstream	Primary team	Responsibilities
Team A – Core fulfilment	Original seven-member team, excluding unavailable payments engineer	Order lifecycle, inventory reservation, picking, dispatch, tracking and support lookup.
Team B – Regional rules and integration	New seven-member team	Regional configuration, tax/shipping rules, payment adapter ownership, refund rule matrix and integration tests.
Shared ownership	Both teams	API contracts, data model, CI pipeline, security, end-to-end tests and release readiness.

Payment engineer replacement: nominate one Team B engineer as payment integration owner, pair with the original team's QA and order developer, and document gateway interfaces, idempotency keys, retry rules, failure states and refund audit fields. Do not rely on undocumented individual knowledge.

Scaled Agile and Lean coordination

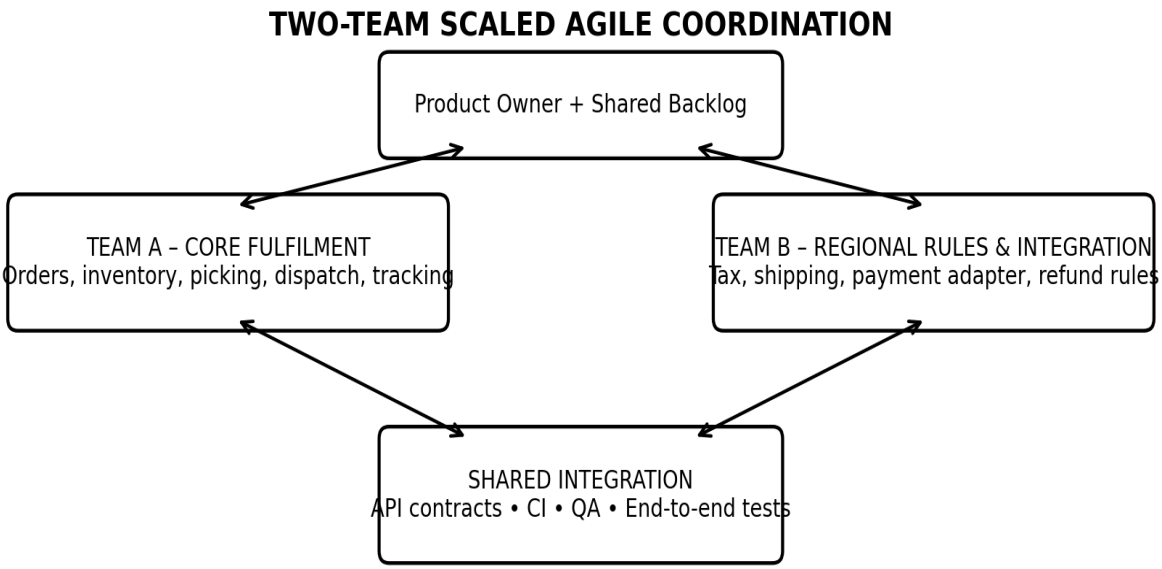


Figure 3. Coordination model for the two delivery teams.

Use a shared product backlog and one Product Owner to prevent teams from optimizing conflicting priorities.

Hold a 30-minute cross-team planning/dependency meeting twice weekly; each team retains its own daily coordination and sprint planning.

Use a shared integration board with dependency links, API contract owners and blocked reasons.
Agree common Definition of Ready, Definition of Done, coding standards, test data and release criteria.
Integrate continuously on a shared test environment; use contract tests to detect incompatible APIs early.
Limit each team's active WIP to 3 stories and shared integration WIP to 2 items; swarm on integration blockers.
Adopt selected SAFe-inspired practices only: shared PI-style eight-week objective, synchronized two-week iterations, cross-team dependency review and integrated system demo. A full SAFe hierarchy is unnecessary for two small teams.

6.4 Revised release sequence

Iteration	Revised focus	Release decision
Sprint 1 completed	Retain the simulated core order and stock foundation.	Reuse validated order and reservation work; correct known defects.
Sprint 2 / Weeks 3–4	Regional data model/configuration; mock payment adapter; basic picking; tax/shipping rule tests.	Gateway integration begins when sandbox access arrives; replacement engineer owns payment work.
Sprint 3 / Weeks 5–6	Region-aware order totals, payment retry, refund approval matrix, dispatch/tracking and support lookup.	Scanner is tested where useful; manual picking remains fallback.
Sprint 4 / Weeks 7–8	Three-region end-to-end validation, reconciliation, accessibility/security regression, defect fixes and release rehearsal.	Defer analytics and advanced automation; release only if critical journey tests pass.

6.5 D. Additional simulated events and comparison

Event	Simulated event after change	Revised response
11	Two warehouses report different tax and shipping rules.	Team B creates region configuration stories and examples.

12	New team discovers its payment engineer is the only member familiar with gateway adapter.	Pair replacement owner with QA; document interface and add contract tests.
13	Regional refund approval differs by warehouse.	Finance defines a region-aware approval matrix; human approval retained.
14	Shared test environment shows incompatible order-region field.	Teams agree one API contract and add migration/contract tests.
15	Integration WIP reaches its limit and blocks a new analytics story.	Analytics is deferred; team swarms to finish regional integration.
16	All three regional synthetic journeys pass the agreed critical checks.	Product Owner accepts the limited release scope subject to final release gate.

Equivalent indicator	**Original Sprint 1 simulation**	**Revised simulation target/result**	**Comparison**
Planned/completed work	4 planned; 2 accepted.	6 critical regional/core stories planned for the iteration; 5 accepted, 1 carried over.	Revised planning uses smaller slices and reserves integration capacity; simulated target is 83%.
Blocked time	1 working day on stock clarification.	0.5 working day average for recorded cross-team blockers after twice-weekly review.	Earlier escalation is expected to reduce waiting.
Cycle time	US01: 3 working days.	Median target: 2.5 working days for small stories after onboarding.	Target only; requires more observations to confirm.

Reopened work	1 story returned for boundary-test defect.	No critical story reopened in the simulated regional acceptance run.	Earlier contract and regional tests are expected to reduce rework.

The revised metrics are illustrative simulation values/targets, not observed delivery data. They are comparable in definition, but the revised period includes different work and team size; conclusions should therefore be treated as directional rather than statistically proven.

6.6 E. Defence of the revised strategy

The redesign is feasible because it increases capacity while reducing scope to the operational journey and splitting work into clear components. Team B absorbs regional rules and payment integration; Team A protects the existing order and fulfilment foundation. Shared contracts, integrated demos and WIP limits reduce coordination waste. The unchanged deadline is protected by deferring analytics and advanced automation. Limitations remain: the replacement payment owner needs rapid onboarding, regional rules may change, and two teams can create integration overhead. The release must be delayed or narrowed rather than bypassing payment safety, refund authorization or critical end-to-end tests.

TASK 7 – INDIVIDUAL TECHNICAL REFLECTION (5 MARKS)

The most significant decision was prioritizing the complete order-to-delivery-to-refund journey over delivering all nine modules. This decision connects the release to the main operational problems: failed orders, inaccurate stock, unclear status and delayed refunds. A broad set of incomplete modules would not solve these problems.

The simulated Sprint 1 evidence showed that clarification and boundary testing consumed more time than expected. Therefore, the strategy was revised to improve backlog refinement, include test cases before work starts, reserve capacity for integration and retain WIP limits. The changed conditions further required region-aware rules, a replacement payment owner and cross-team coordination.

The final design remains vulnerable to gateway instability, incorrect regional tax/shipping configuration, limited stakeholder availability and integration delays. Privacy is addressed by using synthetic data and avoiding real card/customer information. Transparency is supported by recording order/payment/refund changes and keeping human responsibility for refund approvals and stock corrections. Accessibility is included in the Definition of Done through keyboard operation and readable status information.

Before applying this plan to a real team, I would validate regional legal/tax requirements, confirm gateway sandbox and production certification, measure actual team availability, conduct security and accessibility testing, and interview warehouse, support and finance users. I would also collect several sprints of cycle-time and throughput data before making reliable capacity forecasts.

CONCLUSION

The proposed strategy uses Scrum with Lean flow controls and selected XP practices to deliver a focused eight-week release. The backlog prioritizes order creation, stock reservation, payment safety, picking, dispatch, tracking, support lookup and controlled refunds. Simulated Sprint 1 evidence leads to improved refinement and testing practices. The redesign uses two coordinated teams, regional configuration and a replacement payment owner while deferring nonessential features. The release is accepted only when the critical order-to-delivery-to-refund journey is validated.

REFERENCES

1. 1. Rath, S. P., Jain, N. K., Tomer, G., & Singh, A. K. (2025). A systematic literature review of agile software development projects. Information and Software Technology, 182, 107727.
2. 2. Fawzy, A., Tahir, A., Galster, M., et al. (2025). Exploring data management challenges and solutions in agile software development: A literature review and practitioner survey. Empirical Software Engineering, 30, 77.
3. 3. Verwijs, C., & Russo, D. (2024). Do Agile scaling approaches make a difference? An empirical comparison of team effectiveness across popular scaling approaches. Empirical Software Engineering, 29, 75.
4. 4. Dong, H., Dacre, N., Baxter, D., & Ceylan, S. (2024). What is Agile Project Management? Developing a new definition following a systematic literature review. Project Management Journal, 55(6).
5. 5. Anifa, M., Ramakrishnan, S., Kabiraj, S., & Joghee, S. (2024). Systematic review of literature on Agile approach. South Asian Journal of Business and Management Cases, 13(2).
6. 6. Tetteh, S. G. (2024). Empirical study of Agile software development methodologies: A comparative analysis. Asian Journal of Research in Computer Science, 17(5), 30–42.
7. 7. Alqudah, M. K., et al. (2024). A grounded theory of selecting lean and agile practices for software development. Journal of Software: Evolution and Process, 36(4), e2539.